Colatron: End-to-End Text-to-Speech

UE24CS352A Machine Learning, Mini-Project · Team 25, Problem 20 · Mann Mehta (PES2UG24CS266), Marapareddy Deekshitha (PES2UG24CS268) · Repository: github.com/Mann0801/colatron-tts (private)

1. Problem statement

Text-to-speech (TTS) turns written text into audible speech. Our problem revisits the Stanford CS229 (2018) project *End-to-End Text to Speech Synthesis* (Wang, Yang, Li), whose stated problem is to generate an audio file from text input with a seq2seq model with attention, alongside simpler baselines. It trained an SVR, a 4-layer neural network and a seq2seq LSTM with attention on about 200 LJ Speech sentences, mapping words *randomly* to phonemes or letters, and converted predicted spectrograms to audio with an inverse STFT. Their listeners' mean opinion scores (MOS, 1 to 5) were 1.0, 1.7 and 2.5, against 3.82 reported for Tacotron, and the best model repeated words because its attention never aligned. The authors blamed too little data, shallow models, the random text mapping and memory-heavy padding. **Our goal** is to build and train our own seq2seq model with attention from scratch, **Colatron**, with each of these causes fixed; to compare it with a from-scratch baseline that has no attention (reproducing the paper's failure); and to use a pretrained Tacotron 2 only as a reference point.

2. Dataset

Corpus	LJ Speech 1.1: 13,100 clips (23.9 h) of one female speaker reading non-fiction, mono WAV at 22,050 Hz, with transcripts
Colatron	All 13,100 clips: 12,960 train, 100 validation, 40 test. The 40 test clips are the baseline's test clips, so all systems are scored on the same held-out sentences, which Colatron never trains on
Baseline	A 404-clip subset chosen by hashing file names (same on every machine): 324 train, 40 validation, 40 test
Text	Lowercased; numbers, money, ordinals, years and abbreviations expanded ("Dr. Smith paid $42.50 in 1984" becomes "doctor smith paid forty two dollars and fifty cents in nineteen eighty four"); mapped to a **fixed** 37-symbol character vocabulary
Audio	80-band log-mel spectrograms: FFT 1024, window 1024, hop 256, 0 to 8 kHz, slaney scale, natural log. All systems share these settings (src/audio_config.py)

3. Approach

The pipeline has two swappable stages: an **acoustic model** (text to mel spectrogram) and a **vocoder** (mel spectrogram to waveform). Because every component reads and writes the same mel format, any acoustic model can be paired with any vocoder, so each comparison changes only one stage.

Colatron, our main model (trained from scratch). A Tacotron-style seq2seq network with 7.4M parameters. Encoder: character embedding (256-d), three 1-D convolutions (256 channels, kernel 5, batch norm, dropout 0.5) and a bidirectional LSTM (2 x 128). Decoder: a prenet (256, 128; dropout kept on at inference), an attention LSTM (512), *location-sensitive attention* (which sees its previous and cumulative weights, so it moves forward through the text), a decoder LSTM (512), and a linear layer that predicts 3 mel frames per step (reduction factor) plus a stop token. A 5-layer convolutional post-net refines the spectrogram. The loss is L1 + MSE on per-band normalised mels before and after the post-net, binary cross-entropy on the stop token, and a *guided attention loss* (Tachibana et al., 2017) that penalises attention weight far from the diagonal. Training: teacher forcing, Adam (learning rate 1e-3), length-bucketed batches, gradient clipping 1.0, keeping the checkpoint with the lowest validation mel loss: the first 1,000 steps on a laptop CPU (batch 32), the rest on a free Google Colab T4 GPU (batch 64), 5,000 steps and 2.2 h in total. The reduction factor and the guided attention loss are what make attention learn this quickly, which is exactly where the 2018 model failed.

Baseline (trained from scratch, no attention). Character embedding (128-d), three 1-D convolutions (256 channels, kernel 5), linear interpolation of the character features to the target number of frames, and an MLP to 80 mel bands (911k parameters). Each character receives roughly an equal share of time. Trained with L1 loss on normalised mels, Adam (1e-3), clipping 1.0, batch 16, 60 epochs on the 324-clip subset.

Reference and vocoders. A pretrained Tacotron 2 (torchaudio, 28M parameters, trained for days on GPUs on all of LJ Speech) shows what the same model family reaches at full scale. Mels are voiced with Griffin-Lim (iterative phase reconstruction, no learning, our implementation) or a pretrained HiFi-GAN (SpeechBrain). A pretrained wav2vec 2.0 recogniser is used only for evaluation.

4. Implementation overview

Module	What it does	Owner
src/preprocess/	Download LJ Speech, clean text, fixed vocabulary, mel extraction, splits (subset and full corpus), sanity check	Deekshitha, Mann
src/models/baseline.py	Baseline text-to-mel network: training and inference	Deekshitha
src/models/colatron.py	Colatron: model, guided attention loss, training with checkpoints and monitoring, inference	Mann
src/vocoder/, src/models/tacotron.py	Griffin-Lim and HiFi-GAN vocoders; pretrained Tacotron 2 wrapper	Mann
src/infer.py, app/app.py	CLI and Gradio demo with every system, live mel and attention plots	Mann
eval/	DTW mel distance (Deekshitha); recogniser word error rate, also tracked during training (Mann)	Both
src/plot_*.py, src/report_figures.py	Alignment plots and statistics, mel comparisons, training curves	Mann

Python 3.11 and PyTorch 2.14. Everything runs on a laptop CPU; Colatron's training also used a free Colab T4 GPU. The README gives setup and run instructions.

5. Results

Attention learns to align. Colatron's attention forms a diagonal within the first 500 training steps, thanks to the guided attention loss, and then sharpens. On three unseen sentences it is 99 to 100% monotonic (share of decoder steps that never move backwards in the text) and covers 94 to 98% of the characters as the main focus. This is the property the 2018 model lacked, and the reason it repeated words.

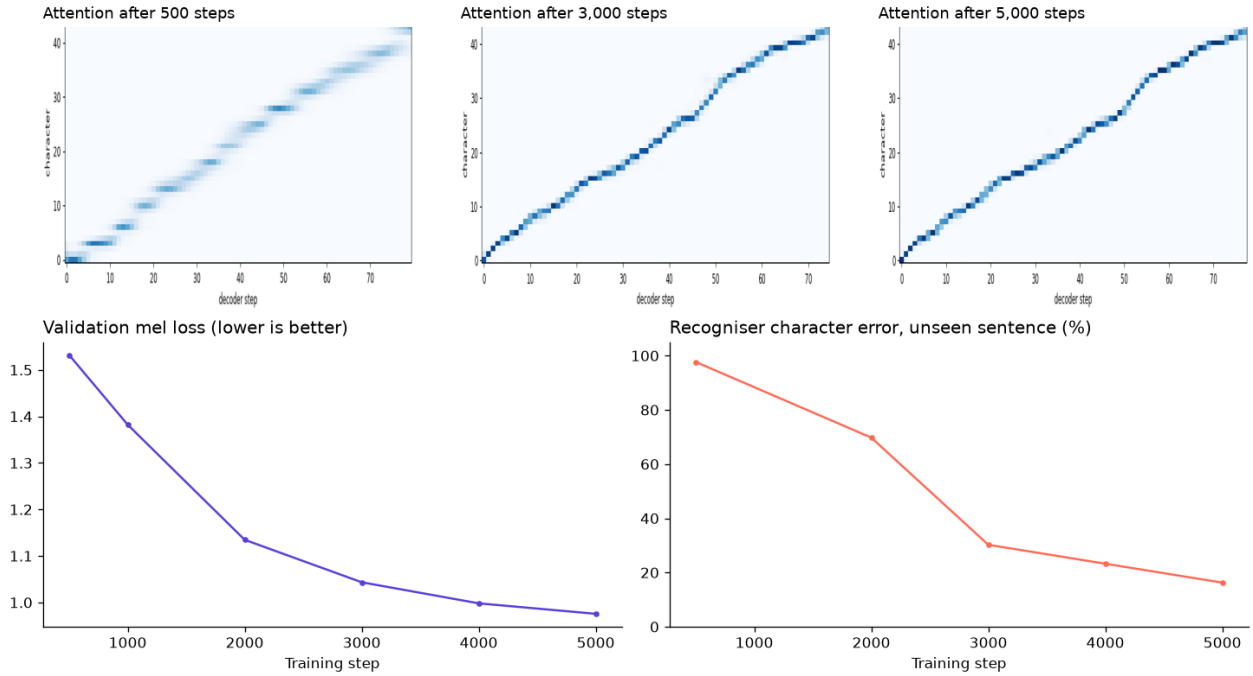


Figure 1: Colatron's attention at three points in training (top), validation mel loss and the recogniser's character error rate on an unseen sentence over training (bottom).

Intelligibility and distance to real speech. We have no human listening test, so a pretrained wav2vec 2.0 recogniser (never trained on LJ Speech) acts as an objective listener: it transcribes each generated clip and we compute the word and character error rates (WER, CER) against the requested sentence. We also convert each output to a mel spectrogram, align it with the real recording's mel by dynamic time warping, and average the distance. All numbers are on the same 20 held-out test clips; lower is better.

System	WER	CER	Mel L1	Mel RMSE
Real recording	5%	2%	-	-
Baseline (ours, no attention) + Griffin-Lim	100%	95%	1.218	1.528
Colatron (ours, from scratch) + Griffin-Lim	29%	12%	0.873	1.151
Colatron (ours, from scratch) + HiFi-GAN	25%	10%	0.887	1.169
Tacotron 2 (pretrained reference) + Griffin-Lim	18%	9%	0.878	1.175

System	WER	CER	Mel L1	Mel RMSE
Tacotron 2 (pretrained reference) + HiFi-GAN	16%	8%	0.901	1.206

With the same vocoder (Griffin-Lim), Colatron is 28% closer to real speech than the baseline in mel L1. On mel distance Colatron is already level with Tacotron 2, but the recogniser still separates them: Tacotron 2 is more intelligible, so word error rate is the more telling metric. Caveat: Tacotron 2 was pretrained on all of LJ Speech, so it may have seen these test clips; Colatron never did.

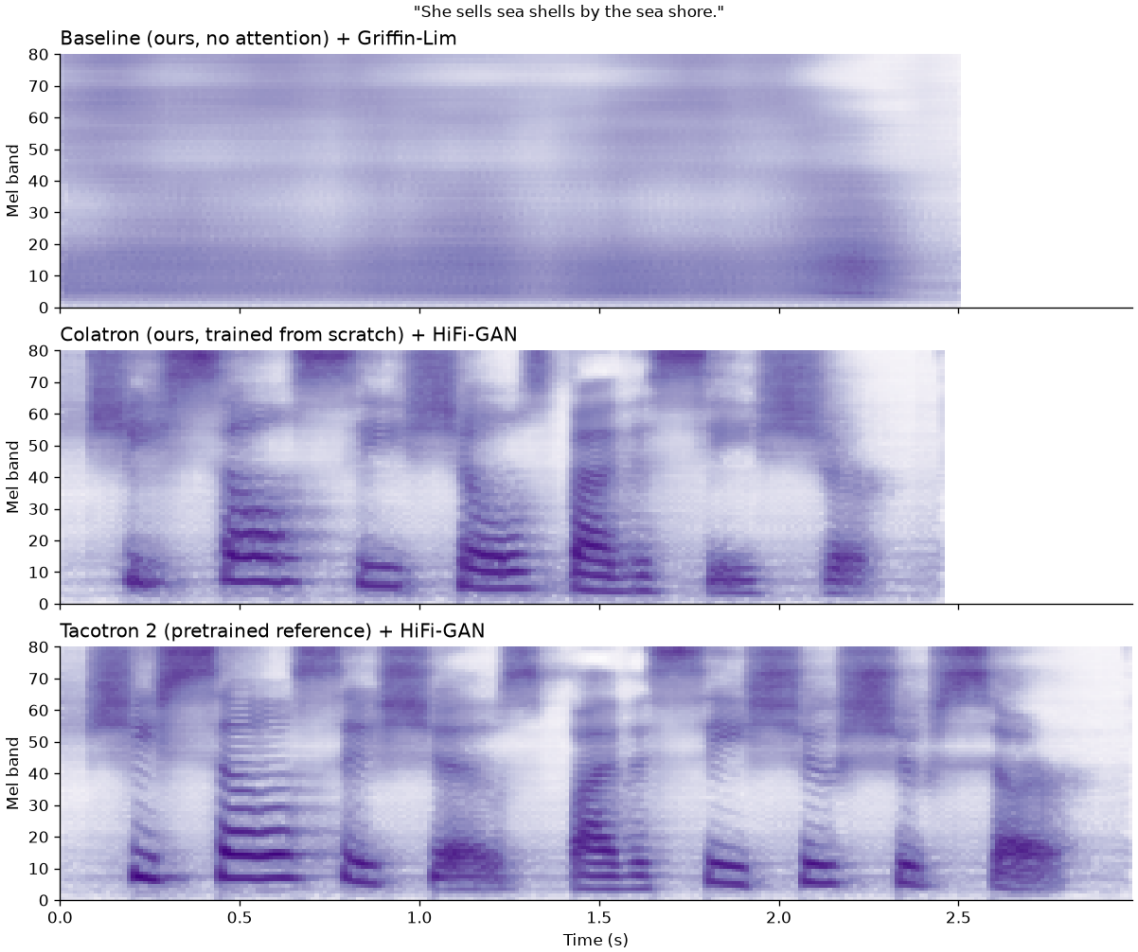


Figure 2: mel spectrograms of the same sentence from the baseline, Colatron and Tacotron 2.

Why the systems sound different. The baseline's mel is a smooth blur with no syllables, pauses or pitch harmonics: with an equal share of time per character, its frames never line up with the real audio, so L1 training drives it to the average spectrum and it sounds like noise. Colatron's attention gives it timing, so syllable structure appears; the pretrained Tacotron 2, four times larger and trained far longer, is sharper still.

Vocoder speed (one 3.3 s sentence, laptop CPU): Griffin-Lim 0.08 s, HiFi-GAN 0.65 s (about 5 times faster than real time), so the demo runs live.

6. Conclusions

• **Attention and data fix the 2018 failure.** Trained from scratch on all 13,100 clips with location-sensitive and guided attention, Colatron reads text in order (99 to 100% monotonic) after only 2.2 h of training on a laptop CPU and a free Colab GPU.
• **Learned timing is essential.** Without attention the baseline reaches 100% word error; Colatron reaches 25% with HiFi-GAN.
• **Scale still matters.** The pretrained Tacotron 2 reaches 16%, against 5% for real recordings.
• **The vocoder decides sound quality.** On identical mels, HiFi-GAN avoids Griffin-Lim's metallic phase artefacts.
• **Limitations.** Colatron trained for only 2.2 h and still sounds robotic; it relies on a pretrained HiFi-GAN vocoder; one speaker and one language; the recogniser is a proxy for human listeners and there is no MOS test. Next steps: train longer on a GPU and run a listening test.